



UNIVERSIDAD DE COLIMA

19/05/2026

PROYECTO DE CHAT ONLINE

PROGRAMACIÓN DISTRIBUIDA DE SERVICIOS DE INTERNET

ALUMNO(S):

GUZMÁN RAMÍREZ ALBERTO ALEXANDER
CRISTIAN OSIRIS RODRIGUEZ CORTES
AMADOR MARTINEZ JESUS ALBERTO

4-B

MAESTRO(A):

AGUAYO ORTUÑO MIGUEL ANGEL

Introducción:

El presente proyecto consiste en el desarrollo de una aplicación web de chat en tiempo real, diseñada para permitir la comunicación instantánea entre múltiples usuarios conectados dentro de una misma red o servidor. La plataforma fue desarrollada utilizando tecnologías modernas orientadas al desarrollo web y la comunicación bidireccional en tiempo real, integrando herramientas como Node.js, Express, Socket.IO y MongoDB.

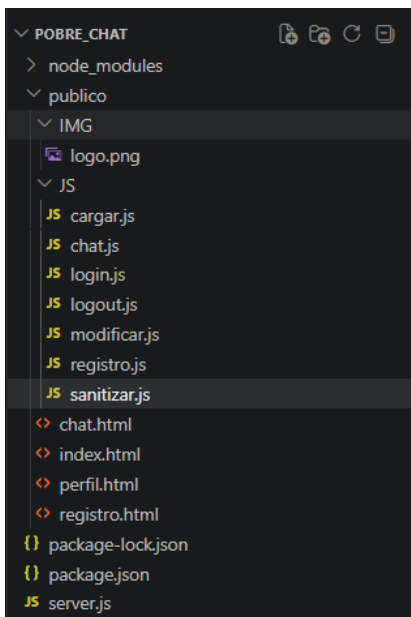
El sistema permite a los usuarios registrarse, iniciar sesión, modificar su perfil y participar en conversaciones grupales mediante una interfaz dinámica y accesible desde distintos dispositivos conectados a la red. La autenticación de usuarios se realiza mediante almacenamiento seguro de contraseñas utilizando algoritmos de hash SHA-256, mientras que la comunicación en tiempo real es administrada mediante WebSockets a través de Socket.IO.

Además, el proyecto incorpora medidas básicas de seguridad orientadas a la protección de la información y prevención de ataques comunes en aplicaciones web, como la sanitización de entradas de usuario, validación de formularios y escape de caracteres HTML para evitar inyecciones de código malicioso.

La arquitectura del sistema está dividida entre cliente y servidor. El servidor administra la lógica principal, autenticación, conexiones activas y distribución de mensajes, mientras que el cliente proporciona una interfaz interactiva desarrollada con HTML, CSS, Bootstrap y JavaScript. Los mensajes del chat son gestionados temporalmente en memoria RAM para optimizar la velocidad de comunicación y reducir el almacenamiento persistente innecesario.

Desarrollo:

Primero creamos el siguiente directorio y archivos:



Desarrollamos el siguiente código:

Server.js:

```
const express = require('express');
const path = require('path');
const mongoose = require('mongoose');
const crypto = require('crypto');
const http = require('http');
const socketIO = require('socket.io');

const app = express();
const puerto = 8080;

// =====
// CREAR SERVIDOR HTTP
// =====
const server = http.createServer(app);

// =====
// SOCKET.IO + CORS
// =====
const io = socketIO(server, {
  cors: {
    origin: "*",
    methods: ["GET", "POST"]
  }
});

// =====
// CONEXIÓN A MONGODB
// =====
mongoose.connect('mongodb://127.0.0.1:27017/Probe_chat')
```

```

        .then(() => console.log("✅ MongoDB conectado (solo para usuarios)"))
        .catch(err => console.log("❌ Error MongoDB:", err));

// =====
// MIDDLEWARES
// =====
app.use(express.static(path.join(__dirname, 'publico')));
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// =====
// FUNCIÓN HASH SHA256
// =====
function sha256(texto) {
    return crypto.createHash('sha256').update(texto).digest('hex');
}

// =====
// MODELO DE USUARIO
// =====
const UsuarioSchema = new mongoose.Schema({
    user_name: {
        type: String,
        unique: true,
        required: true
    },
    user_pass: {
        type: String,
        required: true
    },
    color_code: {

```

```

        type: String,
        default: "#667eea"
    }
});

const Usuario = mongoose.model('Usuarios', UsuarioSchema);

// =====
// RUTA PRINCIPAL
// =====
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'publico', 'index.html'));
});

// =====
// REGISTRO
// =====
app.post('/registro', async (req, res) => {
    try {
        const { user_name, user_pass, confirm_pass, color } = req.body;

        if (!user_name || !user_pass || !confirm_pass) {
            return res.send("Faltan datos");
        }

        if (user_pass !== confirm_pass) {
            return res.send("Las contraseñas no coinciden");
        }

        if (user_name.length < 3) {
            return res.send("El nombre debe tener al menos 3 caracteres");
        }
    }
});

```

```

    }

    const existe = await Usuario.findOne({ user_name });

    if (existe) {
        return res.send("El usuario ya existe");
    }

    const nuevoUsuario = new Usuario({
        user_name,
        user_pass: sha256(user_pass),
        color_code: color || "#667eea"
    });

    await nuevoUsuario.save();

    console.log(`📄 Nuevo usuario registrado: ${user_name}`);

    res.send("Usuario registrado correctamente");

} catch (error) {

    console.log(error);

    res.send("Error en el servidor");
}

});

// =====
// LOGIN
// =====

```

```
app.post('/login', async (req, res) => {

  try {

    const { user_name, user_pass } = req.body;

    if (!user_name || !user_pass) {
      return res.json({
        ok: false,
        msg: "Faltan datos"
      });
    }

    const usuario = await Usuario.findOne({ user_name });

    if (!usuario) {
      return res.json({
        ok: false,
        msg: "Usuario no encontrado"
      });
    }

    const hash = sha256(user_pass);

    if (usuario.user_pass !== hash) {
      return res.json({
        ok: false,
        msg: "Contraseña incorrecta"
      });
    }
  }
});
```

```
        res.json({
            ok: true,
            msg: "Login correcto",
            user_id: usuario._id,
            user_name: usuario.user_name,
            color: usuario.color_code
        });

    } catch (error) {

        console.log(error);

        res.json({
            ok: false,
            msg: "Error en el servidor"
        });
    }
});

// =====
// OBTENER USUARIOS
// =====
app.get('/api/usuarios', async (req, res) => {

    try {

        const usuarios = await Usuario.find({}, '_id user_name color_code');

        res.json(usuarios);

    } catch (error) {
```



```

        res.status(500).json([]);
    }
});

// =====
// CHAT EN MEMORIA
// =====

// Usuarios conectados
const usuariosConectados = new Map();

// Últimos 50 mensajes
let mensajesEnMemoria = [];

// =====
// SOCKET.IO
// =====
io.on('connection', (socket) => {

    console.log(`👤 Nuevo socket conectado: ${socket.id}`);

    // =====
    // USUARIO CONECTADO
    // =====
    socket.on('user-connected', (userData) => {

        usuariosConectados.set(socket.id, {
            userId: userData.userId,
            userName: userData.userName,
            userColor: userData.userColor,

```

```
        socketId: socket.id,  
        connectedAt: new Date()  
    });  
  
    socket.userId = userData.userId;  
    socket.userName = userData.userName;  
    socket.userColor = userData.userColor;  
  
    console.log(`✅ ${socket.userName} conectado`);  
  
    // Historial  
    socket.emit('chat-history', mensajesEnMemoria.slice(-50));  
  
    // Actualizar lista online  
    broadcastOnlineUsers();  
  
    // Avisar a otros  
    socket.broadcast.emit('user-joined', {  
        userId: userData.userId,  
        userName: userData.userName,  
        userColor: userData.userColor,  
        message: `${userData.userName} se ha unido al chat`,  
        usersCount: usuariosConectados.size  
    });  
  
    // Bienvenida  
    socket.emit('welcome', {  
        message: `¡Bienvenido ${userData.userName}!`,  
        usersCount: usuariosConectados.size  
    });  
});
```

```
// =====  
// NUEVO MENSAJE  
// =====  
socket.on('send-message', (messageData) => {  
  
    if (!messageData.text || messageData.text.trim() === '') {  
        return;  
    }  
  
    console.log(`💬 [${socket.userName}]: ${messageData.text.substring(0,  
50)}`);  
  
    const nuevoMensaje = {  
        id: Date.now(),  
        userId: messageData.userId,  
        userName: messageData.userName,  
        userColor: messageData.userColor,  
        text: messageData.text,  
        timestamp: new Date().toISOString()  
    };  
  
    // Guardar en RAM  
    mensajesEnMemoria.push(nuevoMensaje);  
  
    // Limitar a 50 mensajes  
    if (mensajesEnMemoria.length > 50) {  
        mensajesEnMemoria.shift();  
    }  
  
    // Enviar a todos  
    io.emit('new-message', nuevoMensaje);  
}
```

```
});

// =====
// ESCRIBIENDO...
// =====
socket.on('typing', (data) => {

    socket.broadcast.emit('user-typing', {
        userId: socket.userId,
        userName: socket.userName,
        isTyping: data.isTyping
    });
});

// =====
// DESCONEXIÓN
// =====
socket.on('disconnect', () => {

    if (socket.userName) {

        console.log(`👋 ${socket.userName} desconectado`);

        usuariosConectados.delete(socket.id);

        io.emit('user-left', {
            userId: socket.userId,
            userName: socket.userName,
            message: `${socket.userName} ha salido del chat`,
            usersCount: usuariosConectados.size
        });
    }
});
```

```
        broadcastOnlineUsers();
    }
});

});

// =====
// ACTUALIZAR USUARIOS ONLINE
// =====
function broadcastOnlineUsers() {

    const usersList = Array.from(usuariosConectados.values()).map(user => ({
        userId: user.userId,
        userName: user.userName,
        userColor: user.userColor,
        connectedAt: user.connectedAt
    }));

    io.emit('online-users', {
        count: usuariosConectados.size,
        users: usersList
    });
}

// =====
// INICIAR SERVIDOR
// =====
server.listen(puerto, '0.0.0.0', () => {

    console.log(`
```

```

||      🚀 CHAT EN LÍNEA - SERVIDOR ACTIVO      ||
||=====||
||  👤 LOCAL: http://localhost:${puerto}      ||
||  🌐 RED:   http://TU_IP_LOCAL:${puerto}     ||
||                                              ||
||  💾 Usuarios: MongoDB                       ||
||  💬 Mensajes: RAM                           ||
||  👥 Máximo mensajes: 50                     ||
||=====||
||
||      `);
||
||});

```

cargar.js:

```

// =====
// cargar datos
// =====

const user_id = localStorage.getItem("user_id");
const user_name = localStorage.getItem("user_name");
const color = localStorage.getItem("color");

// Si no hay sesión → regresar al login
if (!user_id) {
    window.location.href = "/index.html";
}

// =====
// insertar los datos
// =====

const inputId = document.querySelector("[name=user_id]");
const inputUser = document.querySelector("[name=user_name]");

```

```
const inputColor = document.querySelector("[name=color]");

inputId.value = user_id;
inputUser.value = user_name;
inputColor.value = color;
```

chat.js:

```
// =====
// VARIABLES GLOBALES
// =====

let currentUser = {
    id: null,
    name: null,
    color: "#667eea"
};

let socket = null;
let typingTimeout = null;

// =====
// CARGAR DATOS DEL USUARIO
// =====

function loadUser() {

    const userId = localStorage.getItem('user_id');
    const userName = localStorage.getItem('user_name');
    const userColor = localStorage.getItem('user_color');

    // Si no hay sesión
    if (!userId || !userName) {
        window.location.href = '/';
    }
}
```

```
        return false;
    }

    currentUser.id = userId;
    currentUser.name = userName;
    currentUser.color = userColor || "#667eea";

    console.log("Usuario cargado:", currentUser);

    // Opcionales
    const userNameDisplay = document.getElementById('userNameDisplay');
    const userColorDisplay = document.getElementById('userColorDisplay');

    if (userNameDisplay) {
        userNameDisplay.textContent = currentUser.name;
    }

    if (userColorDisplay) {
        userColorDisplay.style.backgroundColor = currentUser.color;
    }

    return true;
}

// =====
// CONECTAR SOCKET.IO
// =====
function connectSocket() {

    socket = io();
```



```

// =====
// CONEXIÓN EXITOSA
// =====
socket.on('connect', () => {

    console.log('☑ Conectado al servidor');

    // Identificarse
    socket.emit('user-connected', {
        userId: currentUser.id,
        userName: currentUser.name,
        userColor: currentUser.color
    });
});

// =====
// BIENVENIDA
// =====
socket.on('welcome', (data) => {

    addSystemMessage(data.message);

    addSystemMessage(
        `👥 Hay ${data.usersCount} usuario(s) conectado(s)`
    );
});

// =====
// HISTORIAL
// =====
socket.on('chat-history', (history) => {

```

```

        console.log('📖 Historial:', history.length);

        if (history.length === 0) {

            addSystemMessage(
                '💬 No hay mensajes recientes'
            );

        } else {

            renderMessages(history);
        }
    });

    // =====
    // NUEVO MENSAJE
    // =====
    socket.on('new-message', (message) => {

        addMessageToChat(message);
    });

    // =====
    // USUARIO CONECTADO
    // =====
    socket.on('user-joined', (data) => {

        addSystemMessage(
            `💎 ${data.userName} se ha unido (${data.usersCount} conectados)`
        );
    });

```

```

});

// =====
// USUARIO DESCONECTADO
// =====
socket.on('user-left', (data) => {

    addSystemMessage(
        `👋 ${data.userName} salió (${data.usersCount} conectados)`
    );
});

// =====
// LISTA ONLINE
// =====
socket.on('online-users', (data) => {

    updateUsersList(data.users, data.count);
});

// =====
// ESCRIBIENDO
// =====
socket.on('user-typing', (data) => {

    const typingDiv = document.getElementById('typingIndicator');

    if (!typingDiv) return;

    if (
        data.isTyping &&

```

```
        data.userId !== currentUser.id
    ) {

        typingDiv.textContent =

            `🖱️ ${data.userName} está escribiendo...`;

    } else {

        typingDiv.textContent = '';
    }
});

// =====
// ERROR
// =====
socket.on('error', (error) => {

    console.error('Socket error:', error);

    addSystemMessage(`❌ Error: ${error}`);
});

// =====
// DESCONECTADO
// =====
socket.on('disconnect', () => {

    console.log('❌ Desconectado');

    addSystemMessage(

        '❌ Desconectado del servidor'
    );
});
```

```

        );
    });
}

// =====
// RENDER MENSAJES
// =====
function renderMessages(messages) {

    const chatDiv = document.getElementById('chat');

    if (!chatDiv) return;

    chatDiv.innerHTML = '';

    messages.forEach(msg => {

        addMessageToChat(msg, false);

    });

    scrollToBottom();
}

// =====
// AÑADIR MENSAJE
// =====
function addMessageToChat(msg, scroll = true) {

    const chatDiv = document.getElementById('chat');

    if (!chatDiv) return;

```

```
const isOwn = msg.userId === currentUser.id;

const messageDiv = document.createElement('div');

messageDiv.className =
  `message ${isOwn ? 'message-sent' : 'message-received'}`;

// =====
// NOMBRE
// =====
const nameDiv = document.createElement('div');

nameDiv.className = 'message-name';

nameDiv.innerHTML = `
  <span style="color:${msg.userColor || '#667eea'};">
    ${escapeHtml(msg.userName)}
  </span>
`;

// =====
// TEXTO
// =====
const textDiv = document.createElement('div');

textDiv.className = 'message-text';

textDiv.innerHTML =
  parseEmojis(
    escapeHtml(msg.text)
```

```
    );

    // =====
    // HORA
    // =====
    const timeDiv = document.createElement('div');

    timeDiv.className = 'message-time';

    const date = new Date(msg.timestamp);

    timeDiv.textContent = date.toLocaleTimeString([], {
        hour: '2-digit',
        minute: '2-digit'
    });

    // =====
    // AGREGAR
    // =====
    messageDiv.appendChild(nameDiv);
    messageDiv.appendChild(textDiv);
    messageDiv.appendChild(timeDiv);

    chatDiv.appendChild(messageDiv);

    if (scroll) {
        scrollToBottom();
    }
}

// =====
```

```
// MENSAJE SISTEMA
// =====
function addSystemMessage(text) {

    const chatDiv = document.getElementById('chat');

    if (!chatDiv) return;

    const div = document.createElement('div');

    div.className = 'system-message';

    div.textContent = text;

    chatDiv.appendChild(div);

    scrollToBottom();
}

// =====
// USUARIOS ONLINE
// =====
function updateUserList(users, count) {

    const userListDiv = document.getElementById('usersList');

    const onlineCountSpan = document.getElementById('onlineCount');

    if (onlineCountSpan) {
        onlineCountSpan.textContent =
            `Conectados: ${count}`;
    }
}
```



```
}

if (!usersListDiv) return;

usersListDiv.innerHTML = '';

users.forEach(user => {

    const isCurrentUser =
        user.userId === currentUser.id;

    const userItem = document.createElement('div');

    userItem.className = 'user-item';

    userItem.innerHTML = `
        <div class="user-color-dot"
            style="background-color:${user.userColor};">
        </div>

        <div class="user-name">
            ${escapeHtml(user.userName)}
        </div>

        ${isCurrentUser
            ? '<span class="current-user-badge">tú</span>'
            : ''}
    `;

    usersListDiv.appendChild(userItem);
```

```

    });
}

// =====
// ESCAPAR HTML
// =====
function escapeHtml(str) {

    const div = document.createElement('div');

    div.textContent = str;

    return div.innerHTML;
}

// =====
// PARSE EMOJIS
// =====
function parseEmojis(text) {

    return text;
}

// =====
// SCROLL ABAJO
// =====
function scrollToBottom() {

    const chatDiv = document.getElementById('chat');

    if (!chatDiv) return;

```

```
        chatDiv.scrollTop =  
            chatDiv.scrollHeight;  
    }  
  
    // =====  
    // ENVIAR MENSAJE  
    // =====  
    function sendMessage() {  
  
        const input =  
            document.getElementById('mensaje');  
  
        if (!input) return;  
  
        // =====  
        // SANITIZAR  
        // =====  
        let text =  
            Sanitizador.limpiarTexto(input.value);  
  
        // =====  
        // VALIDAR  
        // =====  
        if (!Sanitizador.esValido(text)) {  
  
            alert("Mensaje inválido");  
  
            return;  
        }  
    }
```

```

// =====
// OBJETO MENSAJE
// =====
const messageData = {
    userId: currentUser.id,
    userName: currentUser.name,
    userColor: currentUser.color,
    text: text,
    timestamp: new Date().toISOString()
};

// =====
// ENVIAR
// =====
socket.emit(
    'send-message',
    messageData
);

// =====
// LIMPIAR
// =====
input.value = '';

updateCharCount();

notifyTyping(false);
}

// =====
// NOTIFICAR ESCRITURA

```

```
// =====  
function notifyTyping(isTyping) {  
  
    if (!socket) return;  
  
    socket.emit('typing', {  
        isTyping: isTyping  
    });  
}  
  
// =====  
// MANEJAR ESCRITURA  
// =====  
function handleTyping() {  
  
    notifyTyping(true);  
  
    if (typingTimeout) {  
        clearTimeout(typingTimeout);  
    }  
  
    typingTimeout = setTimeout(() => {  
  
        notifyTyping(false);  
  
    }, 1500);  
}  
  
// =====  
// CONTADOR  
// =====
```

```
function updateCharCount() {

    const input =
        document.getElementById('mensaje');

    if (!input) return;

    const count = input.value.length;

    const max = 200;

    const counterDiv =
        document.getElementById('charCounter');

    if (!counterDiv) return;

    counterDiv.textContent =
        `${count} / ${max}`;

    counterDiv.classList.remove(
        'warning',
        'danger'
    );

    if (count > max * 0.9) {

        counterDiv.classList.add('danger');

    } else if (count > max * 0.7) {

        counterDiv.classList.add('warning');
```

```
    }  
}  
  
// =====  
// AÑADIR EMOJI  
// =====  
function addEmoji(emoji) {  
  
    const input =  
        document.getElementById('mensaje');  
  
    if (!input) return;  
  
    const start = input.selectionStart;  
    const end = input.selectionEnd;  
  
    const text = input.value;  
  
    const newText =  
        text.substring(0, start) +  
        emoji +  
        text.substring(end);  
  
    if (newText.length > 200) {  
  
        alert(  
            "No puedes exceder 200 caracteres"  
        );  
  
        return;  
    }  
}
```

```
        input.value = newText;

        input.setSelectionRange(
            start + emoji.length,
            start + emoji.length
        );

        input.focus();

        updateCharCount();

        handleTyping();
    }

    // =====
    // LOGOUT
    // =====
    function logout() {

        if (socket) {
            socket.disconnect();
        }

        localStorage.clear();

        window.location.href = '/';
    }

    // =====
    // INICIALIZAR
```



```
// =====
document.addEventListener(
  'DOMContentLoaded',
  () => {

    if (!loadUser()) return;

    connectSocket();

    const sendBtn =
      document.getElementById('enviar');

    const msgInput =
      document.getElementById('mensaje');

    const logoutBtn =
      document.getElementById('logout');

    const toggleEmoji =
      document.getElementById('toggleEmoji');

    const emojiPanel =
      document.getElementById('emojiPanel');

    // =====
    // BOTÓN ENVIAR
    // =====
    if (sendBtn) {

      sendBtn.addEventListener(
        'click',
```

```
        (e) => {

            e.preventDefault();

            sendMessage();

        }

    );

}

// =====
// INPUT
// =====

if (msgInput) {

    msgInput.addEventListener(

        'keypress',

        (e) => {

            if (e.key === 'Enter') {

                e.preventDefault();

                sendMessage();

            }

        }

    );

    msgInput.addEventListener(

        'input',

        () => {
```

```
        updateCharCount();

        handleTyping();
    }
};

// =====
// LOGOUT
// =====
if (logoutBtn) {

    logoutBtn.addEventListener(
        'click',
        logout
    );
}

// =====
// PANEL EMOJIS
// =====
if (toggleEmoji && emojiPanel) {

    toggleEmoji.addEventListener(
        'click',
        () => {

            emojiPanel.style.display =
                emojiPanel.style.display === 'none'
                ? 'flex'
                : 'none';
        }
    );
}
```

```

        }

    );

}

// =====
// EMOJIS
// =====

document.querySelectorAll('.emoji-btn')
    .forEach(btn => {

        btn.addEventListener(
            'click',
            () => {

                addEmoji(btn.textContent);

            }
        );

    });

updateCharCount();
}
);

```

login.js:

```

// =====
// LOGIN
// =====

document
    .getElementById("loginForm")
    .addEventListener("submit", async (e) => {

```

```
e.preventDefault();

// =====
// OBTENER INPUTS
// =====
const userInput =
    document.querySelector("[name=user_name]");

const passInput =
    document.querySelector("[name=user_pass]");

if (!userInput || !passInput) {

    alert("Faltan campos");

    return;
}

// =====
// SANITIZAR
// =====
const user_name =
    Sanitizador.limpiarTexto(userInput.value);

const user_pass =
    Sanitizador.limpiarTexto(passInput.value);

// =====
// VALIDAR
// =====
```

```
if (!Sanitizador.esValido(user_name)) {

    alert("Nombre de usuario inválido");

    return;
}

if (!Sanitizador.esValido(user_pass)) {

    alert("Contraseña inválida");

    return;
}

// =====
// DATOS
// =====
const data = {
    user_name,
    user_pass
};

try {

    // =====
    // FETCH LOGIN
    // =====
    const res = await fetch('/login', {

        method: 'POST',
```

```
        headers: {
            'Content-Type': 'application/json'
        },

        body: JSON.stringify(data)
    });

    // =====
    // RESPUESTA
    // =====
    const resultado = await res.json();

    // =====
    // ERROR LOGIN
    // =====
    if (!resultado.ok) {

        alert(resultado.msg);

        return;
    }

    // =====
    // GUARDAR SESIÓN
    // =====
    localStorage.setItem(
        "user_id",
        resultado.user_id
    );

    localStorage.setItem(
```

```

        "user_name",
        resultado.user_name
    );

    localStorage.setItem(
        "user_color",
        resultado.color
    );

    // =====
    // REDIRECCIÓN
    // =====
    window.location.href = "/chat.html";

} catch (error) {

    console.error(error);

    alert(
        "Error al conectar con el servidor"
    );
}

});

```

logout.js:

```

document.addEventListener('DOMContentLoaded', () => {

    const boton = document.getElementById('logout');

    if (boton) {

        boton.addEventListener('click', () => {

            localStorage.removeItem('user_id');

            localStorage.removeItem('user_name');


```



```

        localStorage.removeItem('color');

        alert('sesion cerrada con exito');

        window.location.href = '/index.html';

    });

}

});

```

modificar.js:

```

// =====
// FORMULARIO
// =====

const form = document.querySelector("form");

// =====
// CARGAR USER_ID
// =====

const userIdInput =

    document.querySelector("[name=user_id]");

if (userIdInput) {

    userIdInput.value =

        localStorage.getItem("user_id");

}

// =====
// MODIFICAR PERFIL
// =====

form.addEventListener("submit", async (e) => {

    e.preventDefault();

```

```
// =====  
// OBTENER INPUTS  
// =====  
  
const userId =  
    document.querySelector("[name=user_id]")?.value;  
  
const userNameInput =  
    document.querySelector("[name=user_name]");  
  
const userPassInput =  
    document.querySelector("[name=user_pass]");  
  
const colorInput =  
    document.querySelector("[name=color]");  
  
// =====  
// VALIDAR EXISTENCIA  
// =====  
  
if (  
    !userId ||  
    !userNameInput ||  
    !userPassInput ||  
    !colorInput  
) {  
  
    alert("Faltan campos");  
  
    return;  
}
```

```
// =====  
// SANITIZAR  
// =====  
const user_name =  
    Sanitizador.limpiarTexto(  
        userNameInput.value  
    );  
  
const user_pass =  
    Sanitizador.limpiarTexto(  
        userPassInput.value  
    );  
  
const color =  
    Sanitizador.limpiarTexto(  
        colorInput.value  
    );  
  
// =====  
// VALIDAR  
// =====  
if (!Sanitizador.esValido(user_name)) {  
  
    alert("Nombre inválido");  
  
    return;  
}  
  
// Permitir contraseña vacía  
// si no desea cambiarla  
if (
```

```
        user_pass.length > 0 &&
        !Sanitizador.esValido(user_pass)
    ) {

        alert("Contraseña inválida");

        return;
    }

    // =====
    // VALIDAR COLOR HEX
    // =====
    const colorRegex =
        /^#([0-9A-F]{3}){1,2}$/i;

    if (!colorRegex.test(color)) {

        alert("Color inválido");

        return;
    }

    // =====
    // DATOS
    // =====
    const data = {
        user_id: userId,
        user_name,
        user_pass,
        color
    };
};
```

```
try {

    // =====
    // FETCH
    // =====
    const res = await fetch('/editar', {

        method: 'POST',

        headers: {
            'Content-Type': 'application/json'
        },

        body: JSON.stringify(data)
    });

    // =====
    // RESPUESTA
    // =====
    const resultado =
        await res.json();

    // =====
    // ERROR
    // =====
    if (!resultado.ok) {

        alert(resultado.msg);

        return;
    }
}
```

```
}

// =====
// ACTUALIZAR STORAGE
// =====
localStorage.setItem(
    "user_name",
    resultado.user_name
);

// IMPORTANTE:
// usar user_color
localStorage.setItem(
    "user_color",
    resultado.color
);

// =====
// MENSAJE
// =====
alert("Perfil actualizado");

// =====
// REDIRECCIÓN
// =====
window.location.href =
    "/perfil.html";

} catch (error) {

    console.error(error);
```

```

        alert(
            "Error al actualizar perfil"
        );
    }
});

```

registro.js:

```

// =====
// REGISTRO
// =====

document
    .querySelector("form")
    .addEventListener("submit", async (e) => {

        e.preventDefault();

        // =====
        // OBTENER INPUTS
        // =====

        const userNameInput =
            document.querySelector("[name=user_name]");

        const userPassInput =
            document.querySelector("[name=user_pass]");

        const confirmPassInput =
            document.querySelector("[name=confirm_pass]");

        const colorInput =

```

```
document.querySelector("[name=color]");

// =====
// VALIDAR EXISTENCIA
// =====
if (
    !userNameInput ||
    !userPassInput ||
    !confirmPassInput ||
    !colorInput
) {

    alert("Faltan campos");

    return;
}

// =====
// SANITIZAR
// =====
const user_name =
    Sanitizador.limpiarTexto(
        userNameInput.value
    );

const user_pass =
    Sanitizador.limpiarTexto(
        userPassInput.value
    );

const confirm_pass =
```



```
        Sanitizador.limpiarTexto(
            confirmPassInput.value
        );

const color =
    Sanitizador.limpiarTexto(
        colorInput.value
    );

// =====
// VALIDAR USUARIO
// =====
if (!Sanitizador.esValido(user_name)) {

    alert("Nombre de usuario inválido");

    return;
}

if (user_name.length < 3) {

    alert(
        "El nombre debe tener al menos 3 caracteres"
    );

    return;
}

// =====
// VALIDAR PASSWORD
// =====
```

```
if (!Sanitizador.esValido(user_pass)) {

    alert("Contraseña inválida");

    return;
}

if (user_pass.length < 4) {

    alert(
        "La contraseña debe tener al menos 4 caracteres"
    );

    return;
}

// =====
// VALIDAR CONFIRMACIÓN
// =====
if (user_pass !== confirm_pass) {

    alert(
        "Las contraseñas no coinciden"
    );

    return;
}

// =====
// VALIDAR COLOR HEX
// =====
```

```
const colorRegex =
    /^#([0-9A-F]{3}){1,2}$/i;

if (!colorRegex.test(color)) {

    alert("Color inválido");

    return;
}

// =====
// DATOS
// =====
const data = {
    user_name,
    user_pass,
    confirm_pass,
    color
};

try {

    // =====
    // FETCH
    // =====
    const res = await fetch('/registro', {

        method: 'POST',

        headers: {
            'Content-Type': 'application/json'
```

```
    },

    body: JSON.stringify(data)
  });

  // =====
  // RESPUESTA
  // =====
  const texto = await res.text();

  // =====
  // MENSAJE
  // =====
  alert(texto);

  // =====
  // REDIRECCIÓN
  // =====
  if (
    texto.includes("correctamente")
  ) {

    window.location.href =
      "/index.html";
  }

} catch (error) {

  console.error(error);

  alert(
```

```
        "Error al registrar usuario"

    );

}

});
```

Sanitizar.js:

```
// =====
// SANITIZADOR DE TEXTO
// =====
class Sanitizador {

    // Elimina etiquetas HTML y caracteres peligrosos
    static limpiarTexto(texto) {

        if (!texto) return "";

        // Convertir a string
        texto = String(texto);

        // Eliminar etiquetas HTML
        texto = texto.replace(/<[^>]*>/g, '');

        // Eliminar scripts peligrosos
        texto = texto.replace(/javascript:/gi, '');
        texto = texto.replace(/onerror/gi, '');
        texto = texto.replace(/onclick/gi, '');
        texto = texto.replace(/onload/gi, '');
        texto = texto.replace(/eval\(/gi, '');
        texto = texto.replace(/document\./gi, '');
        texto = texto.replace(/window\./gi, '');

        // Eliminar caracteres peligrosos
        texto = texto.replace(/[<>`"'\\;]/g, '');

        // Eliminar saltos de línea excesivos
        texto = texto.replace(/\n{3,}/g, '\n\n');

        // Eliminar espacios múltiples
        texto = texto.replace(/\s{2,}/g, ' ');
```

```
// Eliminar lenguaje grosero

texto = texto.replace(/sexo/gi, '');
texto = texto.replace(/puto/gi, '');
texto = texto.replace(/puta/gi, '');
texto = texto.replace(/verga/gi, '');
texto = texto.replace(/pito/gi, '');
texto = texto.replace(/pendejo/gi, '');
texto = texto.replace(/pendeja/gi, '');
texto = texto.replace(/estupido/gi, '');
texto = texto.replace(/estupida/gi, '');
texto = texto.replace(/inbecil/gi, '');
texto = texto.replace(/coger/gi, '');
texto = texto.replace(/cogan/gi, '');
texto = texto.replace(/follar/gi, '');
texto = texto.replace(/mierda/gi, '');
texto = texto.replace(/culer/gi, '');
texto = texto.replace(/culo/gi, '');
texto = texto.replace(/teta/gi, '');

// Limitar longitud

texto = texto.substring(0, 200);

// Quitar espacios extremos

texto = texto.trim();

return texto;
}

// Validar longitud

static esValido(texto) {
    if (!texto) return false;
    texto = texto.trim();
    if (texto.length < 1) return false;
    if (texto.length > 200) return false;
    return true;
}
```

```
}  
}
```

chat.html:

```
<!DOCTYPE html>  
<html lang="es" data-bs-theme="dark">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Chat - Modo Oscuro</title>  
  <!-- Bootstrap 5 CSS -->  
  <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"  
rel="stylesheet">  
</head>  
<body class="bg-dark text-light">  
  
  <div class="container py-4">  
    <div class="row justify-content-center">  
      <div class="col-md-9 col-lg-7">  
  
        <!-- Tarjeta Principal del Chat -->  
        <div class="card shadow-lg border-secondary bg-dark">  
  
          <!-- Encabezado con Logo -->  
          <div class="card-header bg-dark py-4 d-flex justify-content-  
between align-items-center border-secondary">  
              
            <div class="btn-group">  
              <a href="perfil.html" class="btn btn-outline-info  
rounded-pill me-2">  
  
                Modificar perfil
```

```

                </a>

                <button id="logout" class="btn btn-outline-danger
rounded-pill">

                    Cerrar sesión

                </button>

            </div>

        </div>

        <!-- Cuerpo del Chat -->

        <div class="card-body bg-black bg-opacity-25 overflow-auto"
id="chat" style="height: 500px;">

            <!-- Los mensajes con estilo oscuro irán aquí -->

            <div class="text-center text-secondary small mt-5">

                Conexión cifrada establecida

            </div>

        </div>

        <!-- Pie de página con el Formulario -->

        <div class="card-footer bg-dark border-secondary p-3">

            <form class="input-group input-group-lg">

                <input id="mensaje" type="text" class="form-control
bg-dark border-secondary text-light shadow-sm" placeholder="Escribe un
mensaje..." aria-label="Mensaje">

                <button id="enviar" class="btn btn-info px-5 shadow"
type="submit">

                    Enviar

                </button>

            </form>

        </div>

    </div> <!-- Fin de la tarjeta -->

</div>

```



```

        </div>

    </div>

    <!-- Bootstrap JS Bundle -->

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

    <script src="JS/logout.js"></script>

    <script src="/socket.io/socket.io.js"></script>

    <script src="JS/sanitizar.js"></script>

    <script src="JS/chat.js"></script>
</body>
</html>

```

index.html:

```

<!DOCTYPE html>

<html lang="es" data-bs-theme="dark">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Login - Modo Oscuro</title>

    <!-- Bootstrap 5 CSS -->

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body class="bg-dark text-light">

    <div class="container d-flex justify-content-center align-items-center vh-100">

        <div class="row w-100 justify-content-center">

            <div class="col-md-6 col-lg-4">

```

```
<!-- Tarjeta de Login -->

<div class="card shadow-lg border-secondary bg-dark p-4">

    <!-- Logo centrado -->

    <div class="text-center mb-4">

    </div>

    <h2 class="text-center mb-4 fw-bold text-info">Login</h2>

    <form id="loginForm">

        <!-- Usuario -->

        <div class="mb-3">

            <label class="form-label">Ingrese su nombre de
usuario</label>

            <input type="text" name="user_name" class="form-
control bg-dark border-secondary text-light" required>

        </div>

        <!-- Contraseña -->

        <div class="mb-4">

            <label class="form-label">Ingrese su
contraseña</label>

            <input type="password" name="user_pass" class="form-
control bg-dark border-secondary text-light" required>

        </div>

        <!-- Botón Iniciar Sesión -->

        <div class="d-grid gap-2 mb-3">

            <button type="submit" class="btn btn-info shadow-sm
fw-bold">Iniciar sesión</button>

        </div>

    </form>
```

```

        <!-- Link de Registro -->
        <div class="text-center">
            <a href="registro.html" class="text-decoration-none
text-info small">
                ¿No tienes cuenta? Crea una cuenta aquí
            </a>
        </div>

    </div> <!-- Fin de la tarjeta -->

</div>
</div>
</div>

<!-- Bootstrap JS Bundle -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.j
s"></script>
<script src="JS/sanitizar.js"></script>
<script src="JS/login.js"></script>
</body>
</html>

```

perfil.html:

```

<!DOCTYPE html>
<html lang="es" data-bs-theme="dark">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Perfil - Modo Oscuro</title>
    <!-- Bootstrap 5 CSS -->

```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-dark text-light">

    <div class="container py-4">
        <div class="row justify-content-center">
            <div class="col-md-9 col-lg-7">

                <!-- Tarjeta Principal de Perfil -->
                <div class="card shadow-lg border-secondary bg-dark">

                    <!-- Encabezado con Logo y Navegación -->
                    <div class="card-header bg-dark py-4 d-flex justify-content-
between align-items-center border-secondary">
                        
                        <div class="btn-group">
                            <a href="chat.html" class="btn btn-outline-info
rounded-pill me-2">
                                Ver chat
                            </a>
                            <button id="logout" class="btn btn-outline-danger
rounded-pill">
                                Cerrar sesión
                            </button>
                        </div>
                    </div>
                </div>

                <!-- Cuerpo: Formulario de Edición -->
                <div class="card-body p-4">
                    <h3 class="mb-4 text-info">Configuración de Perfil</h3>

```

```
<form action="/editar" method="POST">

    <input type="hidden" name="user_id">

    <!-- Nombre de Usuario -->

    <div class="mb-3">

        <label class="form-label text-
secondary">Modificar nombre de usuario</label>

        <input type="text" name="user_name" class="form-
control bg-dark border-secondary text-light" required>

    </div>

    <!-- Contraseña -->

    <div class="mb-3">

        <label class="form-label text-secondary">Cambiar
contraseña</label>

        <p class="small text-muted mb-2">(Si no desea
cambiarla, deje este espacio en blanco)</p>

        <input type="password" name="user_pass"
class="form-control bg-dark border-secondary text-light">

    </div>

    <!-- Selector de Color -->

    <div class="mb-4">

        <label class="form-label text-secondary d-
block">Elige un color para tu nombre:</label>

        <input type="color" name="color" class="form-
control form-control-color bg-dark border-secondary w-100" style="height:
50px;">

    </div>

    <!-- Botón Guardar -->

    <div class="d-grid gap-2">

        <button type="submit" class="btn btn-info
shadow-sm fw-bold">Guardar cambios</button>

    </div>
```

```

        </form>

    </div>

    </div> <!-- Fin de la tarjeta -->

</div>

</div>

</div>

<!-- Bootstrap JS Bundle -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

<script src="JS/sanitizar.js"></script>

<script src="JS/cargar.js"></script>

<script src="JS/modificar.js"></script>

<script src="JS/logout.js"></script>
</body>
</html>

```

registro.html:

```

<!DOCTYPE html>

<html lang="es" data-bs-theme="dark">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Registro - Modo Oscuro</title>

    <!-- Bootstrap 5 CSS -->

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body class="bg-dark text-light">

```

```
<div class="container d-flex justify-content-center align-items-center min-
vh-100 py-5">

  <div class="row w-100 justify-content-center">

    <div class="col-md-8 col-lg-5">

      <!-- Tarjeta de Registro -->

      <div class="card shadow-lg border-secondary bg-dark p-4">

        <!-- Encabezado con Logo y Botón Volver -->

        <div class="d-flex justify-content-between align-items-
center mb-4">

          <a href="index.html">

            <button id="perfil" class="btn btn-outline-info btn-
sm rounded-pill px-3">Iniciar sesión</button>

          </a>

        </div>

        <h2 class="text-center mb-4 fw-bold text-info">Crear
Cuenta</h2>

        <form action="/registro" method="POST">

          <!-- Usuario -->

          <div class="mb-3">

            <label class="form-label">Ingrese su nombre de
usuario</label>

            <input type="text" name="user_name" class="form-
control bg-dark border-secondary text-light" required>

          </div>

          <!-- Contraseña -->

          <div class="mb-3">
```

```

                <label class="form-label">Ingrese su
contraseña</label>

                <input type="password" name="user_pass" class="form-
control bg-dark border-secondary text-light" required>

            </div>

            <!-- Confirmar Contraseña -->

            <div class="mb-3">

                <label class="form-label">Confirme su
contraseña</label>

                <input type="password" name="confirm_pass"
class="form-control bg-dark border-secondary text-light" required>

            </div>

            <!-- Color de Usuario -->

            <div class="mb-4">

                <label class="form-label d-block">Elige un color
para tu nombre:</label>

                <input type="color" name="color" class="form-control
form-control-color bg-dark border-secondary w-100" value="#ff0000"
style="height: 45px;">

            </div>

            <!-- Botón Registrarse -->

            <div class="d-grid gap-2">

                <button type="submit" class="btn btn-info shadow-sm
fw-bold">Registrarse</button>

            </div>

        </form>

    </div> <!-- Fin de la tarjeta -->

</div>

</div>

</div>

```



```
<!-- Bootstrap JS Bundle -->

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

<script src="JS/sanitizar.js"></script>

<script src="JS/registro.js"></script>
</body>
</html>
```

Conclusión:

En conclusión, el desarrollo de este sistema de chat en tiempo real permitió implementar y comprender diferentes tecnologías fundamentales utilizadas en aplicaciones web modernas, integrando tanto el desarrollo del lado del cliente como del servidor. A través del uso de Node.js, Express, Socket.IO y MongoDB, se logró construir una plataforma funcional capaz de gestionar usuarios, autenticar sesiones y mantener comunicación instantánea entre múltiples clientes conectados.

El proyecto demostró la importancia de la comunicación en tiempo real mediante WebSockets, permitiendo que los mensajes fueran transmitidos de manera rápida y eficiente sin necesidad de recargar la página. Asimismo, se aplicaron conceptos esenciales de seguridad informática, como la sanitización de datos, validación de entradas y protección contra ataques, contribuyendo a mejorar la integridad y seguridad de la aplicación.

Repositorio:

https://github.com/alex1175-hub/pobre_chat